

WA3850 Lab Guide

GitHub and GitHub Codebase Management for Leaders



Part of
Accenture

© 2026 Ascendient, LLC

Revision 1.0.0 published on 2026-05-08

No part of this book may be reproduced or used in any form or by any electronic, mechanical, or other means, currently available or developed in the future, including photocopying, recording, digital scanning, or sharing, or in any information storage or retrieval system, without permission in writing from the publisher.

Trademark Notice: Product or corporate names may be trademarks or registered trademarks, and are used only for identification and explanation without intent to infringe.

To obtain authorization for any such activities (e.g., reprint rights, translation rights), to customize this book, or for other sales inquiries, please contact:

Ascendient, LLC

1 California Street Suite 2900

San Francisco, CA 94111

<https://www.ascendientlearning.com>

USA: 1-877-517-6540, email: getinfousa@ascendientlearning.com

Canada: 1-877-812-8887 toll free, email: getinfo@ascendientlearning.com

Lab Materials and Environment Guidelines

Ascendient Learning is pleased to provide lab materials and environments as part of your learning experience. To ensure a safe and effective learning experience for everyone:

● You may:

- Use the lab environment for your own coursework.
- Save any important files or data from the environment to your local computer or online storage (if allowed by your employer).
- Retain lab files for your personal use after the class ends.

● You may not:

- Share your lab materials or environment with anyone outside of your course.
- Store any personal, private, confidential, or proprietary company information within the lab environment.
- Rely on the lab environment for data storage, as all work is permanently deleted after each class.
- Misuse the lab environment for any illegal, malicious, or non-class-related activities.

🚨 Important

The lab environment is under Ascendient Learning's control and is not a part of your organization's IT ecosystem. Do not place your organization's information into the lab environment and always follow your organization's policies and procedures for working with IT systems.

Table of Contents

1. Evergreen Quote Pull Request.....	5
1.1. Why This Lab Exists.....	6
1.2. Open the Starter in VS Code.....	7
1.3. Initialize a Git Repository.....	8
1.4. Sign in to Your Provided GitHub Account.....	9
1.5. Create an Empty GitHub Repository.....	9
1.6. Push the Local Repo to GitHub.....	10
1.7. Create a Feature Branch.....	12
1.8. Edit the Hero Heading.....	12
1.9. Commit and Push the Feature Branch.....	13
1.10. Open the Pull Request.....	14
1.11. Self-Review the Pull Request.....	15
1.12. Squash and Merge.....	17
1.13. Pull the Merge Back Down.....	17
1.14. What You Just Did.....	18
1.15. Bonus: Open the PR from the gh CLI.....	19
1.16. Review.....	20

Evergreen Quote Pull Request

LAB 1

1.1. Why This Lab Exists

Your lab Virtual Machine (VM) has a prepared starter version of the Evergreen Insurance quote page — HTML, CSS, Flexbox, a small JavaScript file. Today it is simply a small app in a folder on your laptop. Every change to it so far has lived entirely on your VM.

That is not how code gets into a real codebase. Every change typically enters a repository through a **pull request** or **PR**: an author proposes a set of edits on a branch, a reviewer reads the diff and leaves comments, Continuous Integration (CI) runs tests automatically, and only after approval does the change merge into `main`. You sit in PR-related conversations every day — standups, sprint reviews, post-incident retrospectives. Understanding what each part of a PR page means turns those conversations from vague to specific.

This lab walks you end-to-end through that cycle for the first time:

- Take the Evergreen project and push it to a fresh GitHub repository of your own.
- Create a short-lived branch with one small change — rewording the hero heading.
- Open a pull request against `main`, fill out a real description (Summary, Why, Test plan), and self-review the diff.
- Approve the PR and **Squash and merge** it.
- Pull the merge back down to your local `main` and confirm the history is clean.

You may not be authoring pull requests as part of your day job after this week. You **will** be reading them — dozens per sprint. The muscle memory of having opened one yourself, even once, makes every future PR you read easier to parse.

 Tip

The pedagogical win this week is **the workflow**, not the change itself. The hero heading tweak is a placeholder for "any one-line edit." The ritual — branch, commit, push, open PR, describe, review, merge, pull back — is the sequence to recognize.

1.2. Open the Starter in VS Code

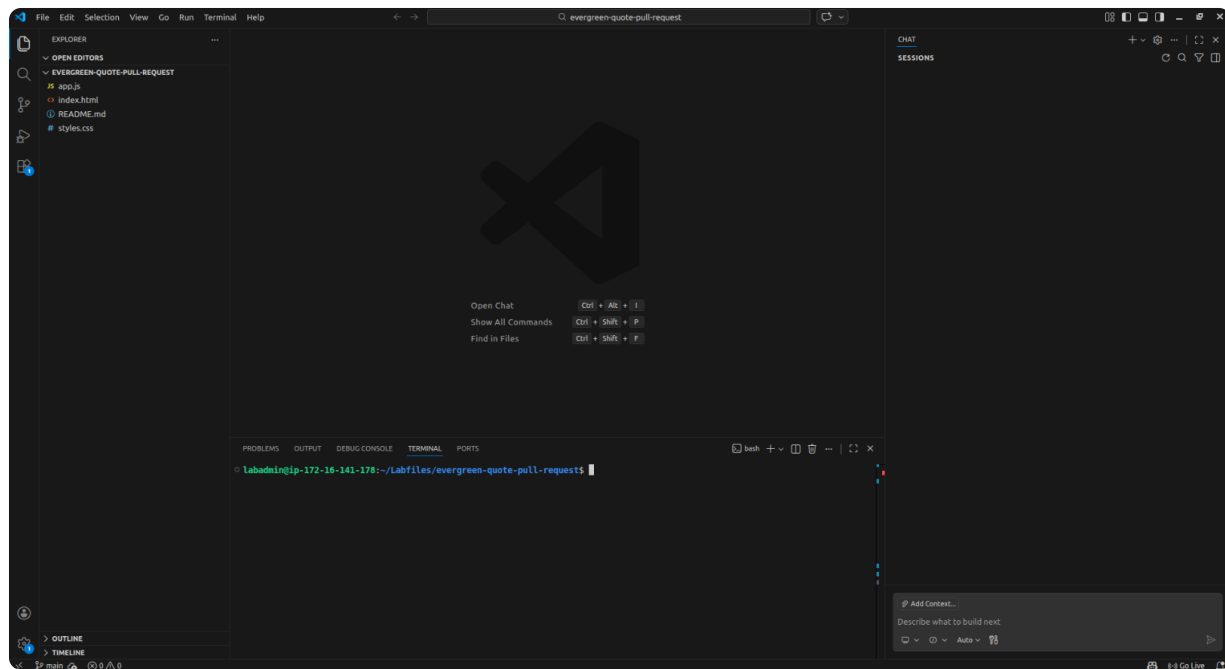
1. Launch VS Code from the lab VM's applications menu (or the desktop shortcut).
2. In VS Code, choose **File → Open Folder**, navigate to `Labfiles/evergreen-quote-pull-request` in your home folder, and click **Open**.
3. Open the VS Code integrated terminal via **Terminal → New Terminal** (or press `Ctrl+``). It opens at the bottom of the VS Code window, already pointed at the lab folder. Confirm with:

```
pwd
```

You should see a path ending in `Labfiles/evergreen-quote-pull-request`.

The project opens with three familiar files plus a README:

- `index.html` — Evergreen page structure (provided for you)
- `styles.css` — the stylesheet (provided for you)
- `app.js` — the quote calculation script with the Auto rate set to 1.5 (provided for you)



1.3. Initialize a Git Repository

1. In the VS Code terminal, execute the command below to confirm the folder's Git status:

```
git status
```

If you see output starting with `On branch main` or similar — good, the folder is already a repository and you can skip ahead to **Sign in to Your Provided GitHub Account**.

2. If instead you see `fatal: not a git repository`, in the VS Code terminal, execute the commands below to initialize one:

```
git init
git add .
git commit -m "Initial Evergreen quote page"
```


The first command creates the hidden `.git` folder that holds all version history. The second stages every current file. The third records the first commit.

Note

If `git commit` complains about a missing author identity, in the VS Code terminal, execute the commands below to set it once:

```
git config user.email "your.name@example.com"
git config user.name "Your Name"
```

...and re-run the commit.

1. In the VS Code terminal, execute the command below to confirm the branch name is `main` (newer Git installs default to this; older ones may say `master`):

```
git branch --show-current
```

2. If it says `master`, in the VS Code terminal, execute the command below to rename the branch:

```
git branch -M main
```

1.4. Sign in to Your Provided GitHub Account

1. Open your browser (Firefox or Chrome — either works) and navigate to <https://github.com>.
2. Sign in with your provided GitHub account credentials.

1.5. Create an Empty GitHub Repository

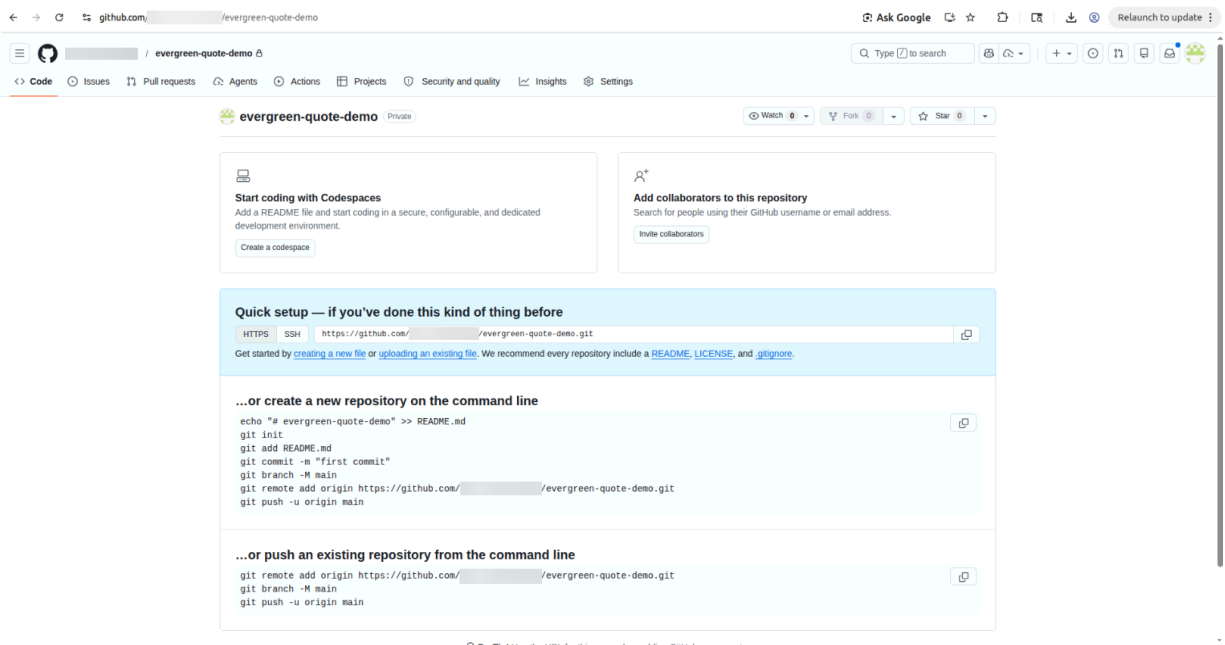
1. Click the **+** icon in the top-right toolbar and choose **New repository**.

2. Fill in the form:

- **Repository name:** evergreen-quote-demo
- **Description:** Practice repo for Phase 1, Week 2 – pull request walkthrough
- **Visibility:** select **Private**.

3. Click **Create repository**.

You land on an empty-repo page with a block of commands titled "...or push an existing repository from the command line." That block is what you will run next.



1.6. Push the Local Repo to GitHub

GitHub shows two possible remote URLs — HTTPS and SSH. Use HTTPS for this lab (GitHub will prompt for a browser sign-in or a Personal Access Token the first time you push).

1. Back in the VS Code terminal, from `~/Labfiles/evergreen-quote-pull-request`, run:

```
git remote add origin https://github.com/YOUR-USERNAME/  
evergreen-quote-demo.git  
git branch -M main  
git push -u origin main
```

Replace `YOUR-USERNAME` with your actual GitHub username (the URL on the GitHub page has the right value; copy it verbatim).

2. The first `git push` may prompt for GitHub credentials. Follow the prompts. If it opens a browser tab for you to click **Authorize**, do that and return to the VS Code terminal.

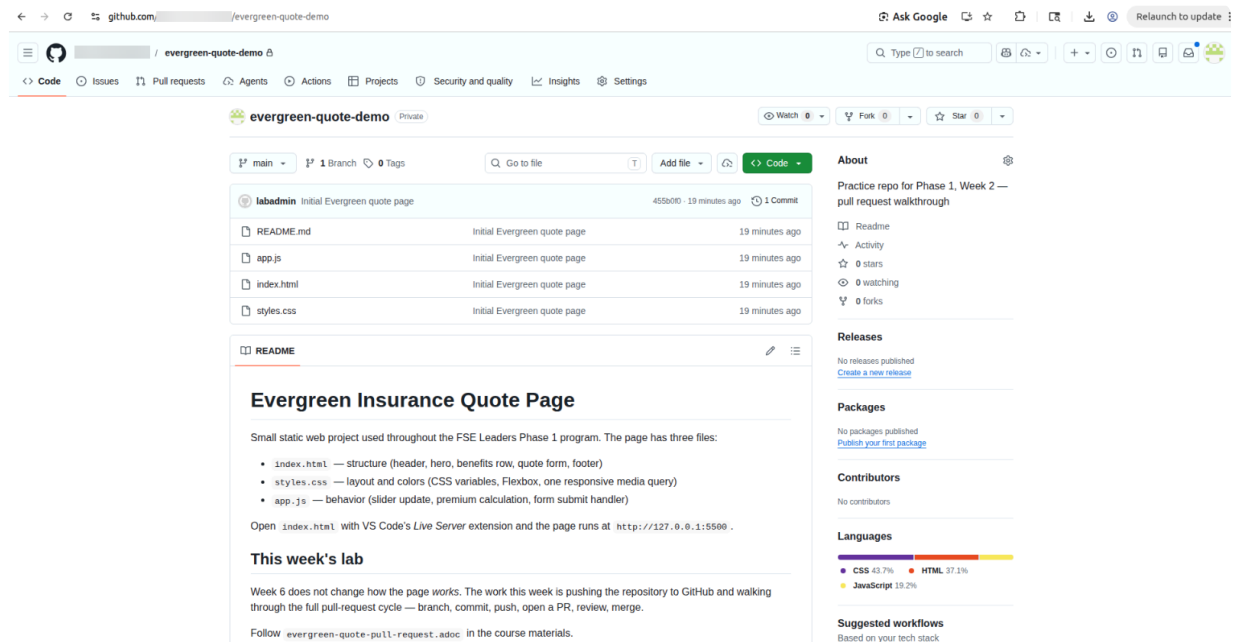
Tip

If HTTPS authentication fails repeatedly, the easiest fallback is the `gh` CLI (pre-installed on your lab VM). In the VS Code terminal, execute the command below:

```
gh auth login
```

...then re-run the `git push`. `gh auth login` handles credential storage for you.

1. Refresh the GitHub page in your browser. The `index.html`, `styles.css`, `app.js`, and `README.md` files are now visible in the repo. The default branch is `main` and shows one commit.



1.7. Create a Feature Branch

1. Back in the VS Code terminal, create and switch to a feature branch:

```
git checkout -b update-hero-copy
```

The branch name — `update-hero-copy` — describes the change. Many teams use naming conventions like `feature/...`, `bugfix/...`, or a ticket-tracker prefix. For this lab a simple descriptive name is fine.

2. In the VS Code terminal, execute the command below to confirm the branch:

```
git branch --show-current
```

You see `update-hero-copy`.

1.8. Edit the Hero Heading

1. Open `index.html` in VS Code. Around line 22, find the hero heading:

```
<section class="hero">
  <h1>Coverage that grows with you</h1>
  <p class="tagline">Get a personalized quote in under a
minute.</p>
</section>
```

2. Change the `<h1>` text to:

```
<section class="hero">
  <h1>Coverage that scales with your life</h1>
  <p class="tagline">Get a personalized quote in under a
minute.</p>
</section>
```

3. Save the file (use **File → Save**, or press `Ctrl+S` — if the shortcut is captured by the browser, the menu always works). If you have Live Server running, the page in the browser refreshes automatically and you see the new heading.

Tip

This is a trivial text change on purpose. The copy tweak is a stand-in for "any one-line edit." The point this week is **not** the words on the page; it is the ritual that gets the words on the page.

1.9. Commit and Push the Feature Branch

1. In the VS Code terminal, stage and commit:

```
git add index.html
git commit -m "Update hero heading to 'scales with your life'"
```

2. In the VS Code terminal, execute the command below to push the new branch to GitHub. The first push of a new branch needs the `-u` flag to set the upstream:

```
git push -u origin update-hero-copy
```

GitHub confirms the push and often prints a URL you can click to open the compare-and-pull-request page directly.

3. Refresh the GitHub repo in your browser. A yellow banner and a green button appear at the top: **"update-hero-copy had recent pushes — Compare & pull request."**

1.10. Open the Pull Request

1. Click **Compare & pull request**. You land on the Open-a-pull-request form. The **base** dropdown should say `main`; the **compare** dropdown should say `update-hero-copy`. Confirm both.
2. Note that the title field is pre-filled with the commit message from your last commit. You can edit it if you want, but for this lab, leave it as-is.
Update hero heading to 'scales with your life'

3. For the body, write a real PR description — this is the single most valuable habit you can carry out of this lab. Use the following template:

Summary

Rewords the hero heading on the Evergreen quote landing page from "Coverage that grows with you" to "Coverage that scales with your life."

Why

The new copy reads as more active and connects the product to the customer's life stage rather than implying a fixed growth trajectory. Aligns with recent marketing guidance on voice.

Test plan

- Opened ``index.html`` with Live Server locally; confirmed hero heading renders correctly at desktop and mobile widths.
- No JavaScript or CSS changes; the benefits row and the quote form continue to render unchanged.
- ``git diff main..update-hero-copy`` shows exactly one hunk (line 22 of ``index.html``).

That is a real PR description. The Summary tells a reviewer **what** in one sentence. The Why tells them **why** — the single most important line in the description. The Test plan tells them how the author convinced themselves it works.

4. Scroll down on the form. The **Files changed** section previews your diff. Green lines indicate additions, red lines indicate deletions. Spot-check that the diff is what you expect.
5. Click **Create pull request**.

The screenshot shows the GitHub 'Comparing changes' interface. At the top, it says 'Comparing changes' and 'Choose two branches to see what's changed or to start a new pull request. If you need to, you can also [compare across forks](#) or [learn more about diff comparisons](#).' Below this, there are two branches selected: 'base: main' and 'compare: update-hero-copy'. A green checkmark indicates 'Able to merge. These branches can be automatically merged.' The main section is 'Add a description', which has a 'Write' tab and a 'Preview' tab. The 'Write' tab is active, showing a description with three sections: 'Summary', 'Why', and 'Test plan'. The 'Summary' section contains the text: 'Rewords the hero heading on the Evergreen quote landing page from "Coverage that grows with you" to "Coverage that scales with your life."' The 'Why' section contains the text: 'The new copy reads as more active and connects the product to the customer's life stage rather than implying a fixed growth trajectory. Aligns with recent marketing guidance on voice.' The 'Test plan' section contains a list of bullet points: '- Opened "index.html" with Live Server locally; confirmed hero heading renders correctly at desktop and mobile widths.', '- No JavaScript or CSS changes; the benefits row and the quote form continue to render unchanged.', and '- "git diff main...update-hero-copy" shows exactly one hunk (line 22 of "index.html").' To the right of the description, there are sections for 'Reviewers' (with a 'Request' button), 'Copilot' (with a 'Request' button), 'Projects' (None yet), 'Milestone' (No milestone), 'Development' (with a 'Use Closing keywords' link), and 'Helpful resources' (with a 'GitHub Community Guidelines' link). At the bottom right, there is a green 'Create pull request' button, which is highlighted with a red box.

1.11. Self-Review the Pull Request

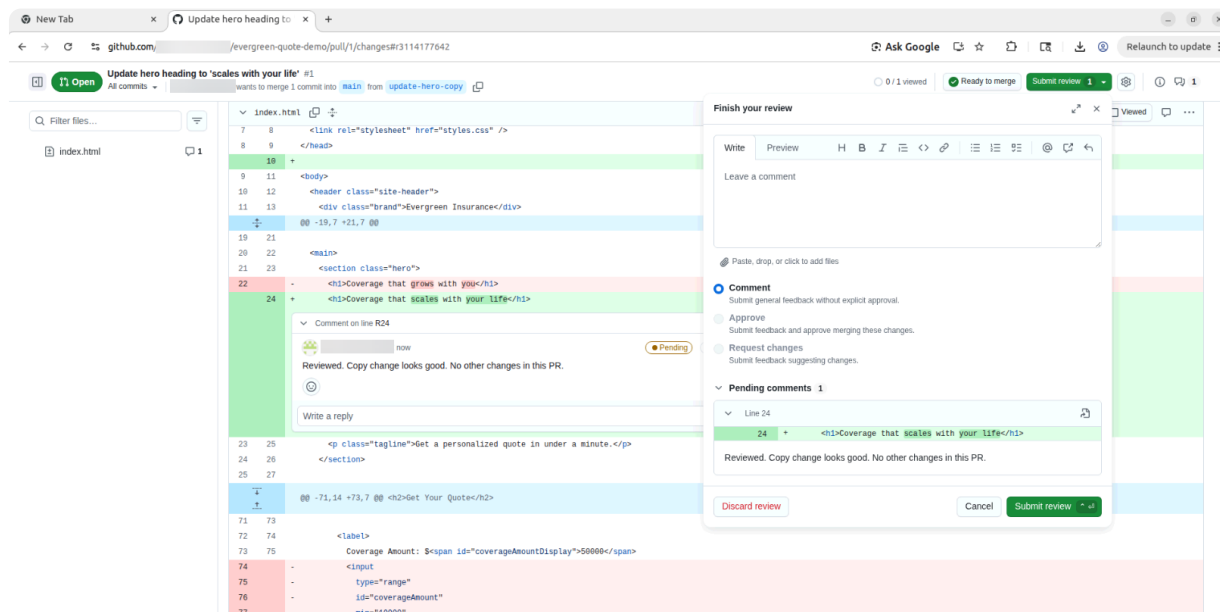
You land on the PR page proper. Take a moment to absorb the layout:

- Tabs: **Conversation, Commits, Checks, Files changed**
- Right sidebar: Reviewers, Assignees, Labels, Projects, Milestone
- Bottom of the page: merge controls
 - i. Click the **Files changed** tab. The diff renders in the standard two-color format (red for deletions, green for additions).
 - ii. Hover over any line in the diff. A blue + icon appears in the gutter. Click it. A comment box opens. Type a comment:

Reviewed. Copy change looks good. No other changes in this PR.

iii. Submit the comment with **Start a review**, then go to the top-right corner and click **Submit review**. Three radio buttons appear:

- **Comment** — no approval weight
- **Approve** — counts toward branch protection's approval requirement (disabled on your own PRs)
- **Request changes** — blocks the merge
 - i. Notice that **Approve** is greyed out with the message "**Pull request authors can't approve their own pull requests.**" That is a hard GitHub rule — it applies to every PR on every repo, regardless of branch protection. Select **Comment**, leave the comment field blank (or add a short note), and click **Submit review**.



1. Switch back to the **Conversation** tab (if not already there). Your review now shows in the timeline as a comment. Because this repo has no branch protection requiring approvals, the merge button at the bottom is already green and ready to use.

Note

On a real team, the PR author usually never approves their own PR. A teammate reviews the diff and clicks **Approve**, and branch protection rules typically require at least one such approval before the merge button unlocks. The mechanics you just practiced (opening **Files changed**, leaving an inline comment, submitting a review) are exactly what that teammate does — the only difference is which radio button they get to click at the end.

1.12. Squash and Merge

1. Scroll to the merge controls at the bottom of the **Conversation** tab. Next to the green **Merge pull request** button is a small dropdown arrow. Click it.
2. Three options appear:
 - **Create a merge commit** — preserves the branch's commit history as a merge commit on `main`.
 - **Squash and merge** — flattens all the branch's commits into one clean commit on `main`.
 - **Rebase and merge** — replays the branch's commits on top of `main` one by one, no merge commit.

Select **Squash and merge**. The button text changes. Click it.

3. A confirmation box appears with the squash commit message (GitHub pre-fills it from the PR title). Leave it as-is or tweak it, then click **Confirm squash and merge**.
4. The PR turns purple. A **Delete branch** button appears — click it. This is the button that saves every repository from accumulating dozens of stale branches over time.

1.13. Pull the Merge Back Down

1. Back in the VS Code terminal, switch to the `main` branch and pull the latest:

```
git checkout main
git pull
```

The `git checkout main` switches your local clone back to the `main` branch (which, on your laptop, has not changed yet — it still reflects pre-merge state). `git pull` fetches the latest commits from GitHub and fast-forwards your local `main` to include the squash-merged change.

2. In the VS Code terminal, execute the command below to confirm:

```
git log --oneline
```

You see two commits: the original "Initial Evergreen quote page" and a new "Update hero heading to 'scales with your life'" — the squash commit. The feature branch commits are not in the log because the squash flattened them.

3. Open `index.html` in VS Code once more. The hero heading now reads **"Coverage that scales with your life"** — the merge is reflected in your working copy too.
4. Your local feature branch still exists. In the VS Code terminal, execute the command below to clean it up:

```
git branch -d update-hero-copy
```

1.14. What You Just Did

You just completed the full code-change ritual, end-to-end:

1. Pushed a local project to a fresh GitHub repository.
2. Branched off `main` for an isolated change.
3. Committed the change on the branch, not on `main`.
4. Pushed the branch to GitHub and opened a pull request.
5. Wrote a real PR description with Summary, Why, and Test plan.
6. Self-reviewed the diff in the Files changed tab.
7. Left an inline comment.
8. Submitted the self-review as a **Comment** (GitHub blocks authors from approving their own PRs).

9. Squash-merged the PR into `main` and deleted the branch.
10. Pulled the merge back down to your local `main`.

❗ Important

Every one of those steps happens dozens of times a week on any active codebase. The engineers you partner with will have the entire sequence as muscle memory. You do not need to have it as muscle memory — but you need to recognize every step when you see one. You now do.

1.15. Bonus: Open the PR from the `gh` CLI

Five extra minutes of practice, if you want them. The `gh` CLI is already installed on your lab VM.

1. In the VS Code terminal, create and push a new branch with another trivial edit (change the `<footer>` copy, for example), then open a PR directly from the command line:

```
git checkout -b footer-copy-tweak
# ... edit index.html (can be any trivial change), save ...
git add index.html
git commit -m "Tweak footer copy"
git push -u origin footer-copy-tweak
# note the use of gh and not git for the next step.
# run gh auth login if you haven't already authenticated with
GitHub from the CLI.
gh pr create --fill
```

`gh pr create --fill` uses your branch name and most recent commit message to fill in the PR title and body automatically. Good for scratch PRs; not a substitute for writing a real description when the change matters.

2. Note the PR number at the end of the output. You will need it for the next step. Your number may differ from the one shown below. `image::assets/the`

terminal output of `gh pr create` showing the new PR URL and number.png[The terminal output of `gh pr create` showing the new PR URL and number,800]

3. In the VS Code terminal, execute the command below to close the PR from the `gh` CLI (replace `<PR_NUMBER>` with the actual number of the PR you just created):

```
gh pr close <PR_NUMBER> --delete-branch
```

1.16. Review

You did one thing this week, and it stacks with everything before:

- You took a project from your laptop to a real GitHub repository.
- You opened a real pull request with a real description.
- You self-reviewed a diff, left an inline comment, approved, and merged.

The next time an engineer says "can you review this PR?" — you know what to look at. The next time someone mentions "branch protection blocked the merge" — you know what that means. The next time a PR description says "Test plan: ..." — you recognize the template.

Tip

Make it a habit to read one PR on a repo you care about every day for the next two weeks. You will be surprised how fast your fluency grows. A PR a day is ten hours a week of engineering context for the cost of ten minutes of reading.